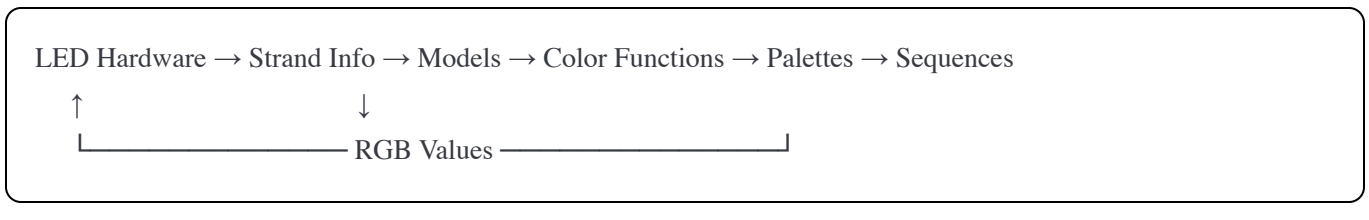
LED Controller Architecture Guide

Overview

This LED controller system creates dynamic lighting effects by connecting several key components: **strand definitions**, **color functions**, **palettes**, **models**, and **sequences**. Understanding how these interact is essential for creating and modifying lighting shows.

System Architecture

The Data Flow



1. **Physical LEDs** are organized into strands
2. **Strand Info** maps physical LEDs to geometric edges
3. **Models** assign color functions to edges
4. **Color Functions** generate colors using palettes
5. **Sequences** orchestrate models over time
6. **RGB values** are sent back to the physical LEDs

Core Components

1. Strand Info (ledconstants.h)

Purpose: Defines the physical LED hardware configuration.

Key Elements:

- `numberofpins`: How many output pins control LED strands
- `ledsperstrip`: Maximum LEDs per strand (typically 260)
- `stranddata[][][]`: Maps LED positions to geometric edges

Strand Data Format:

```
cpp

{length, edgeNumber, direction}
```

- `length`: Number of LEDs in this segment
- `edgeNumber`: Which geometric edge (0-119 for a 24-cell)
- `direction`: Forward (1) or reverse (-1)

Example:

```
cpp

const int stranddata[numberofpins][maxedgesperstrand][3] = {
  {
    {40, 0, 1},  // 40 LEDs on edge #0, forward direction
    {40, 1, 1},  // 40 LEDs on edge #1, forward direction
    {4, 7, 1},   // 4 LEDs on edge #7 (spacer/transition)
    {40, 2, 1},  // 40 LEDs on edge #2, forward direction
  }
};
```

Designer Actions:

- Modify strand mappings when changing physical wiring
- Adjust `ledsperstrip` for different strand lengths

- Update `numberofpins` and `pinList[]` for different hardware

---

2. Color Functions (colorfunctions.h/cpp)

Purpose: Generate colors based on position along an edge (0.0 to 1.0).

Types:

Stateless Functions (simple, no memory):

- `rainbow(position)`: HSV rainbow based on position
- `constantlyDark(position)`: Returns black
- `breathingColor(position)`: Pulsing effect

Stateful Functions (complex, maintain state over time):

- `Fire2012ColorFunction`: Simulated fire effect
- `AudioReactiveColorFunction`: Responds to sound
- `PlasmaColorFunction`: Flowing plasma animation
- `FFTFireColorFunction`: Audio-reactive fire using FFT analysis
- `CylonColorFunction`: Scanning light effect
- `VUMeterColorFunction`: Audio level visualization
- `ParticlesColorFunction`: Moving particle system
- `BeatDetectColorFunction`: Detects and visualizes beats
- `SimpleColorViewer`: Displays palette gradients

Palette Support: Most stateful functions accept a palette parameter:

```
cpp

Fire2012ColorFunction("fire2012", 55, 120, false, "ocean")
                                                  ^^^^^^-- palette name
```

Registration: Functions are registered by index in `colorfunctions.cpp`:

```
cpp

registerStatefulColorFunction(3, new Fire2012ColorFunction("fire2012"));
registerStatefulColorFunction(5, new AudioReactiveColorFunction("audio"));
```

Designer Actions:

- Create new color functions by inheriting from `StatefulColorFunction`
- Register new functions in `initializeStatefulColorFunctions()`
- Modify existing function parameters (cooling, sparking, sensitivity)
- Note slot assignments in header comments (indices 0-14+ are allocated)

---

3. Palettes (paletteregistry.h/cpp)

Purpose: Define color schemes that color functions use.

How They Work:

- Palettes are gradients with 16 color stops
- Color functions sample these gradients at different positions
- One palette can be used by multiple functions simultaneously

Available Palettes:

Custom Palettes (defined in paletteregistry.cpp):

- `fire`: Black → red → orange → yellow → white
- `ocean`: Dark blue → cyan → light blue

- `forest`: Dark green → light green → yellow-green
- `sunset`: Purple → red → orange → yellow
- `rainbow`: Full spectrum colors
- `lava`: Black → dark red → orange → yellow → white
- `ice`: Black → dark blue → light blue → white
- `plasma`: Blue → purple → magenta → orange → yellow
- `audiogreen`: Dark green → bright green → yellow (for audio viz)
- `audiospectrum`: Blue → teal → green → yellow (frequency mapping)
- `white`: Pure white for testing

Built-in FastLED Palettes:

- `heat`, `party`, `cloud`, `ocean_builtin`, `forest_builtin`, `rainbow_builtin`, `rainbowstripe`

Palette Definition Example:

```cpp
DEFINE_GRADIENT_PALETTE( CustomFire_gp ) {
  0,  0,  0,  0,   // Position 0: Black (R, G, B)
 64, 255,  0,  0,   // Position 64: Red
128, 255,255,  0,   // Position 128: Yellow
255, 255,255,255    // Position 255: White
};

// Then register it:
PaletteRegistry::registerPalette("fire", CustomFire_gp);
```

Designer Actions:

- Create new palettes using `DEFINE_GRADIENT_PALETTE` macro
- Register palettes in `PaletteRegistry::initialize()`
- Palettes are automatically available to all functions
- Change function palettes at runtime using `switchPalette()`

---

4. Models (models.h/cpp)

Purpose: Map color functions to geometric edges, creating a complete lighting pattern.

Structure: A model contains:

- A name (for reference in sequences)
- Edge models array: 120 edges, each with 6 parameters:

```cpp
[functionIndex, startOffset, scale, unused, unused, unused]
```

- `functionIndex`: Which color function to use (0-99)
- `startOffset`: Starting position on the gradient (0-10000)
- `scale`: How much of the gradient to use (typically 10000 = full)

Simple Example:

```cpp
// Create a model where all edges show rainbow
std::array<std::array<int, 6>, 120> testModel;
for(int i = 0; i < 120; i++) {
    testModel[i] = {1, 0, 10000, 0, 0, 0};  // Function #1 (rainbow), full gradient
}
colormodel test(testModel, "test");
```

```cpp
// Different functions on different edges
testModel[0] = {3, 0, 10000, 0, 0, 0};   // Edge 0: fire2012
testModel[1] = {5, 0, 10000, 0, 0, 0};   // Edge 1: audio reactive
testModel[2] = {1, 0, 10000, 0, 0, 0};   // Edge 2: rainbow
// ... configure remaining edges
```

Edge Permutations: Models can be transformed using edge permutations (geometric rotations/reflections):

```cpp
colormodel* rotated = baseModel->applyEdgePermutation(rotationPerm, "rotated");
```

Model Merging: Combine two models to create hybrid effects:

```cpp
colormodel* hybrid = colormodel::mergeModels(model1, model2, "hybrid");
```

Designer Actions:

- Define base models in `hdlo_models.cpp`
- Use edge permutations to create variations without redefining
- Merge models for complex multi-function effects
- Models are automatically registered and available by name

---

5. Sequences (sequences.h/cpp, modelsequence.h/cpp)

Purpose: Orchestrate models over time with transitions and audio synchronization.

Structure: A sequence consists of:

- **Multiple named sequences** in a registry
- Each sequence has **multiple steps**
- Each step specifies:
  - A model name
  - Color functions (with optional palette overrides)
  - Duration in seconds
  - Transition type (INSTANT, FADE, WIPE)
  - Transition duration

Basic Sequence Example:

```cpp
```

```cpp
void initializeSequences(modelsequence* seq) {
    seq->clearRegistry();

    // Start a new sequence
    seq->startNewSequence("Morning Show", 120, true);

    // Add steps
    seq->addStep("flowoctahedron", {
        "dark",             // Function 0: no light
        "fire2012",         // Function 1: fire effect
        "plasma",           // Function 2: plasma
        "rainbow"           // Function 3: rainbow
    }, 10, FADE, 2);        // 10 sec duration, 2 sec fade

    seq->addStep("cycle", {
        "dark",
        "audio",
        "particles",
        "cylon"
    }, 15, FADE, 3);
}
```

Inline Palette Override: You can specify palettes per-function within a step:

```cpp
seq->addStep("test", {
    "dark",
    {"fire2012", "ocean"},      // Fire effect with ocean palette
    {"plasma", "sunset"},       // Plasma with sunset palette
    {"simplecolor", "lava"}     // Show lava gradient
}, 20, FADE, 2);
```

Audio Configuration: Audio sources are now set per-sequence, not per-step:

```cpp
// Pure microphone input
seq->startNewSequence("Live Audio", 60, true,
                      AudioSourceConfig(AUDIO_MICROPHONE));

// SD card playback (loops automatically)
seq->startNewSequence("Music Sync", 60, true,
                      AudioSourceConfig(AUDIO_SD_CARD, "song.wav"));

// Microphone with auto-fallback to SD card
seq->startNewSequence("Adaptive", 60, true,
                      AudioSourceConfig("ambient.wav", 5000));
                      // Switches to ambient.wav after 5 sec of silence
```

Multiple Sequences: The system can hold multiple sequences and cycle through them:

```cpp
seq->startNewSequence("Sequence 1", 60);
seq->addStep(...);
seq->addStep(...);

seq->startNewSequence("Sequence 2", 45);
seq->addStep(...);
seq->addStep(...);

// System automatically cycles: Sequence 1 (60s) → Sequence 2 (45s) → repeat
```

Designer Actions:

- Define sequences in `sequences.cpp`
- Add steps with model names and function arrays

- Use inline palette syntax `{"function", "palette"}` for variety
- Set sequence-level audio configuration
- Enable/disable sequences with the `enabled` flag
- Adjust transition types and durations for smooth or dramatic changes

---

How It All Connects

Example: Complete Flow

1. **Physical Setup**: 260 LEDs on pin 24, mapped to 6 edges (40 LEDs each) with spacers
2. **Strand Definition** (ledconstants.h):

```cpp
const int stranddata[1][10][3] = {
  {
    {40, 0, 1}, {4, 7, 1},
    {40, 1, 1}, {4, 7, 1},
    {40, 2, 1}, {4, 7, 1},
    {40, 3, 1}, {4, 7, 1},
    {40, 4, 1}, {4, 7, 1},
    {40, 5, 1}
  }
};
```

3. **Palette Creation** (paletteregistry.cpp):

```cpp
DEFINE_GRADIENT_PALETTE( Twilight_gp ) {
  0,  32,  0,  64,   // Deep purple
 128, 255, 32,  0,   // Orange
 255, 255,192, 64    // Light yellow
};
PaletteRegistry::registerPalette("twilight", Twilight_gp);
```

4. **Model Definition** (hdlo_models.cpp):

```cpp
std::array<std::array<int, 6>, 120> eveningModel;
for(int i = 0; i < 120; i++) {
  eveningModel[i] = {3, 0, 10000, 0, 0, 0};  // All edges: fire2012
}
colormodel evening(eveningModel, "evening");
```

5. **Sequence Creation** (sequences.cpp):

```cpp
seq->startNewSequence("Evening Show", 180, true,
                      AudioSourceConfig("lounge.wav"));

seq->addStep("evening", {
  "dark",
  {"fire2012", "twilight"},   // Fire with twilight palette
  {"plasma", "twilight"},
  {"breathing"}
}, 30, FADE, 3);
```

6. **Runtime**:
   - Controller reads strand info to know where LEDs are
   - Sequence tells model to use fire2012 with twilight palette
   - Fire2012 function generates heat values at each position
   - Twilight palette converts heat to purple→orange→yellow colors

- Colors are sent to physical LEDs through the OctoWS2811 controller

---

Quick Reference for Designers

Adding a New Palette

1. Define gradient in `paletteregistry.cpp` using `DEFINE_GRADIENT_PALETTE`
2. Register in `PaletteRegistry::initialize()`
3. Use by name in sequences: `{"function", "newpalette"}`

Adding a New Color Function

1. Create class inheriting from `StatefulColorFunction` in `colorfunctions.h`
2. Implement `getColor(position)`, `updateState()`, and `reset()`
3. Register in `initializeStatefulColorFunctions()` in `colorfunctions.cpp`
4. Note the index number and add to slot comments
5. Use by name in sequences

Creating a New Model

1. Define edge array (120 edges × 6 parameters)
2. Create `colormodel` object in `hdlo_models.cpp`
3. Model auto-registers and becomes available by name
4. OR: Use edge permutations to derive from existing models

Building a Sequence

1. Edit `sequences.cpp`
2. Call `seq->startNewSequence(name, duration)`
3. Add steps with `seq->addStep(model, {functions}, duration, transition)`
4. Use `{"function", "palette"}` syntax for inline palette overrides
5. Set audio config in `startNewSequence()` for the whole sequence

Modifying Strand Layout

1. Edit `stranddata` in `ledconstants.h`
2. Update `numberofpins`, `ledsperstrip`, and `pinList[]` if needed
3. Recompile and upload

LLM Code Overview Comment

```cpp
```

```
/*****************************************************************************
* LED CONTROLLER SYSTEM ARCHITECTURE - Quick Reference for LLM Analysis
*
* CORE COMPONENTS:
*
* 1. STRAND INFO (ledconstants.h)
*    - Physical LED mapping: pins → strands → edges
*    - stranddata[pin][segment][3] = {length, edgeNum, direction}
*    - Example: {40,0,1} = 40 LEDs on edge 0, forward
*
* 2. COLOR FUNCTIONS (colorfunctions.h/cpp)
*    - Generate colors from position (0.0-1.0) + time/audio
*    - Stateless: rainbow, constantlyDark, breathingColor
*    - Stateful: Fire2012, Audio, Plasma, FFTFire, Cylon, VUMeter, etc.
*    - Registered by index (0-99) in statefulColorFunctions[]
*    - Most support palette parameter for color schemes
*
* 3. PALETTES (paletteregistry.h/cpp)
*    - 16-stop gradient definitions: DEFINE_GRADIENT_PALETTE(name_gp) {...}
*    - Registered: PaletteRegistry::registerPalette("name", name_gp)
*    - Available: fire, ocean, forest, sunset, rainbow, lava, ice, plasma, etc.
*    - Functions sample palettes: ColorFromPalette(palette, position)
*
* 4. MODELS (models.h/cpp)
*    - Map functions to geometric edges (120 edges for 24-cell)
*    - edgemodels[120][6] = {funcIndex, startPos, scale, 0, 0, 0}
*    - Example: {3, 0, 10000, 0, 0, 0} = function #3, full gradient
*    - Can be transformed via edge permutations or merged
*    - Auto-register in global registry, accessible by name
*
* 5. SEQUENCES (sequences.h/cpp, modelsequence.h/cpp)
*    - Orchestrate models over time with transitions
*    - Multiple named sequences in registry, auto-cycle
*    - Per-sequence audio: microphone, SD card, or auto-fallback
*    - Steps: {modelName, [functions], duration, transition}
*    - Inline palette override: {"function", "palette"}
*
* DATA FLOW:
*    Hardware → StrandInfo → Models → ColorFunctions → Palettes → Sequences
*                                ↓
*                           RGB Values → Hardware
*
* KEY CONCEPTS:
* - Position: 0.0-1.0 float representing location along an edge
* - Edge: Geometric line segment in the structure (0-119 for 24-cell)
* - Function Index: Integer (0-99) identifying a color function
* - Palette Name: String identifying a color gradient
* - Model Name: String identifying a complete edge→function mapping
* - Transition: INSTANT, FADE, or WIPE between sequence steps
*
* TYPICAL MODIFICATION TASKS:
* 1. New palette: Define in paletteregistry.cpp, register in initialize()
* 2. New function: Subclass StatefulColorFunction, register in colorfunctions.cpp
* 3. New model: Define edge array in hdlo_models.cpp
* 4. New sequence: Edit sequences.cpp, use existing models/functions/palettes
* 5. Hardware change: Update ledconstants.h stranddata array
*
* ACTUAL FILES TO CHECK:
* - Strands: ledconstants.h
* - Functions: colorfunctions.h (declarations), colorfunctions.cpp (registration)
* - Palettes: paletteregistry.h (interface), paletteregistry.cpp (definitions)
* - Models: models.h (interface), hdlo_models.cpp (definitions)
* - Sequences: sequences.h (interface), sequences.cpp (choreography)
* - Orchestration: modelsequence.h/cpp (sequence engine)
*****************************************************************************/
```

Common Patterns

Pattern 1: Audio-Reactive Show

```cpp
seq->startNewSequence("Audio Show", 180, true,
                      AudioSourceConfig(AUDIO_MICROPHONE));

seq->addStep("flowoctahedron", {
    "dark",
    {"audio", "fire"},      // React to full spectrum
    {"vumeter", "ocean"},   // Show audio levels
    {"beatdetect", "sunset"} // Highlight beats
}, 30, FADE, 2);
```

Pattern 2: Palette Showcase

```cpp
seq->addStep("test", {
    "dark",
    {"simplecolor", "fire"},
    {"simplecolor", "ocean"},
    {"simplecolor", "forest"},
    {"simplecolor", "sunset"},
    {"simplecolor", "lava"}
}, 60, FADE, 3);
```

Pattern 3: Animated Effects

```cpp
seq->addStep("cycle", {
    "dark",
    {"fire2012", "lava"},   // Realistic fire
    {"plasma", "plasma"},   // Flowing plasma
    {"particles", "rainbow"}, // Moving particles
    {"cylon", "ice"}        // Scanning effect
}, 45, FADE, 2);
```

Pattern 4: Model Variations

```cpp
// Base model
colormodel base(baseEdges, "base");

// Rotated version using edge permutation
colormodel* rotated = base.applyEdgePermutation(rotation1, "rotated");

// Merged version combining two models
colormodel* hybrid = colormodel::mergeModels("base", "other", "hybrid");
```

Summary

The LED controller is a layered system where:

1. **Hardware configuration** (strand info) stays mostly static
2. **Palettes and functions** are defined once and reused extensively
3. **Models** map functions to geometry (with possible transformations)
4. **Sequences** provide high-level choreography and are the main creative focus

For most design work, you'll spend your time in `sequences.cpp` combining existing models, functions, and palettes in creative ways. Adding new palettes is simple. Creating new color functions or models requires more work but greatly expands possibilities.

The inline palette syntax `{"function", "palette"}` in sequences gives you enormous flexibility without modifying underlying code, making it the most powerful tool for rapid experimentation and design iteration.